



FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

TEST 1

DATE: 27 MARCH 2025

TIME: 1 ½ Hour

EXAMINER: BOAMAH-ABU, CHARLES (PhD)

TOTAL MARKS: 50

NO. OF PAGES: 6 (Including cover page)

INSTRUCTIONS/REQUIREMENTS

1. *This examination is closed book*
2. *Answer all questions*
3. *Where stated "Briefly answer", please give a short answer*
4. *Where stated "Write the missing parts" of code, please write only the missing parts*
5. *Ensure your handwriting is readable*

Question 1

5 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false)

1. A concrete class can contain an abstract method.
2. Only a class which contains an abstract method can be declared abstract.
3. An interface can extend another interface.
4. An interface can contain instance variables.
5. Design patterns are only guidelines not solutions to common software problems.

Question 2

4 marks

Briefly answer the following.

1. “Integrative programming supports interoperability”. Explain. (2)
2. What is the template pattern used for? (2)

Question 3

10 marks

1. The class **X** (below) illustrates **method overload**. What is wrong with the code? (2)

```
package q3;

public class X {
    private int a() {
        return 0;
    }
    public double a() {
        return 0;
    }
}
```

2. Class **Z** inherits abstract class **Y**. The code below illustrates **method override**. What is wrong with the code in **Z**? (3)

```
package q3;

public abstract class Y {
    public abstract double a();
}
```

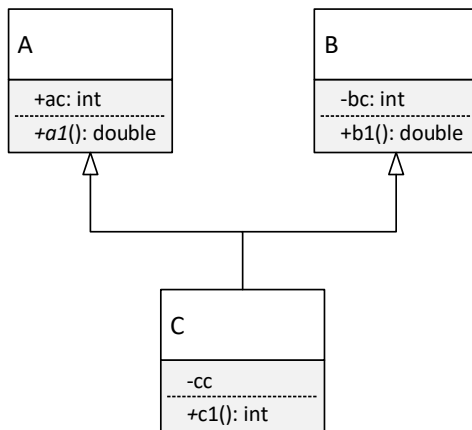
```
package q3;

public class Z extends Y{
    private int a() {
        return 0;
    }
}
```

3. A concrete class **C** implements an interface **I**. In each of the following, state whether the statement is **valid** or **invalid**. Explain your answer.
- `I i1=new I();` (2)
 - `C c1=new C();` (1)
 - `I i2=new C();` (2)

Question 4 5 marks

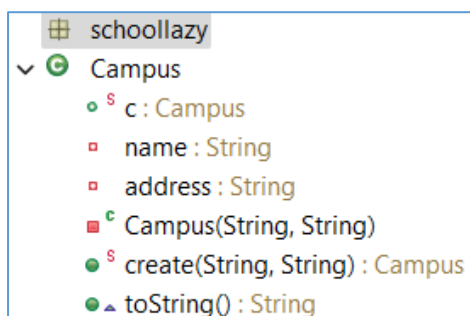
The UML diagram below shows class **C** inherits classes **A** and **B**.



- What is the problem with the diagram that you cannot write the Java code? (2)
- If **A** contains a static variable and an abstract method and **B** contains an instance variable and method implementation, explain how you can resolve the problem (**do not redraw the diagram**). (3)

Question 5 9 marks

A school has only one campus. Below is the structure of a **Singleton** class **Campus** (based on the **lazy initialization**). **name** and **address** are instance variables.



1. Write the missing parts of the class **Campus** as follows (**write only the missing parts**).
 - 1.1. Complete the declaration of the variable **c**. (1)
 - 1.2. Complete the method **create**, which takes two string parameters (a **name** and **address**) and returns **Campus** (based on **lazy initialization**). (6)

```
package schoollazy;
public class Campus {
    _____ Campus c;

    private String name;
    private String address;

    private Campus(String name, String address) {
        this.name = name;
        this.address = address;
    }

    //method to create Campus
    _____ create(_____ ) {
        _____
        _____
        _____
    }

    public String toString() {
        return "Name: "+name +"\tAddress: "+ address;
    }
}
```

2. The class **Tester** (below) contains the method **main**, which creates an instance (**c1**) of **Campus** with the values "Joshua High" and "111 Nowhere". Explain why the statement is incorrect. (2)

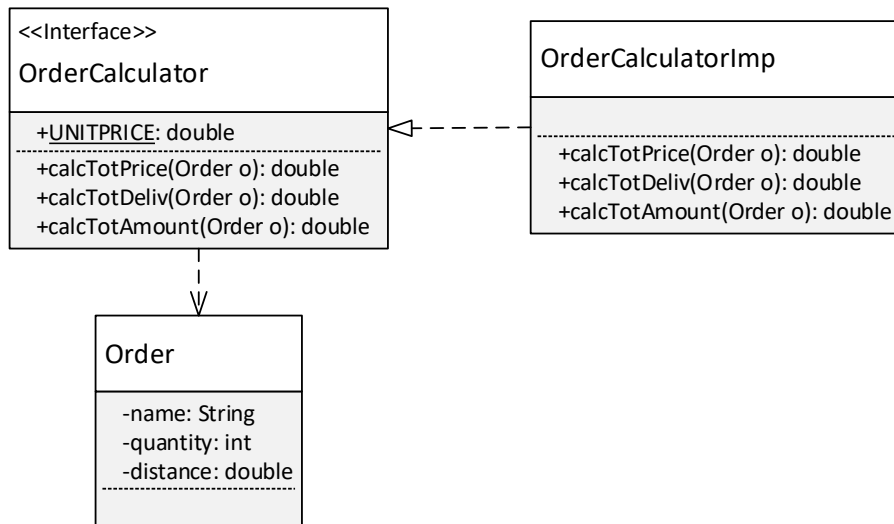
```
package schoollazy;

public class Tester {

    public static void main(String[] args) {
        Campus c1=new Campus("Joshua High", "111 Nowhere");
    }

}
```

NLM Ltd sells and delivers blocks to customers. The unit price of a block is R10.00. The UML diagram below shows a class **Order**, an interface **OrderCalculator** and a class **OrderCalculatorImp**, which implements the interface.



- Order** consists of the **name** of the customer, the **quantity** of blocks to buy and the **distance** for delivery.
- OrderCalculator** consists of:
 - UNITPRICE**, the unit price of a block (a constant, R10.00)
 - calcTotPrice(Order o)** takes an **Order** as a parameter and returns the **total price of blocks**. This is calculated based on the **unit price** and **quantity** of block
 - calcTotDeliv(Order o)** takes an **Order** as a parameter and returns the **total delivery fee**. This is calculated as follows: delivery is **free for the first 10km**; if the distance is more than 10km, **each additional kilometer is charged at R5**.
 - calcTotAmount(Order o)** takes an **Order** as a parameter and returns the **total amount payable** for the order (i.e., the **total price of blocks** and **total delivery fee**). This is calculated by calling the methods above.

1. Write the code for the interface.

(5)

```

package order;

_____ OrderCalculator {
_____
_____
_____
_____
}
  
```

2. Complete the missing parts of the class **OrderCalculatorImp** as follows (write only the missing code)

2.1 Complete the class header. (1)

2.2 Complete the method **calcTotDeliv**. (5)

```
package order;

public class OrderCalculatorImp _____ {

    @Override
    public double calcTotPrice(Order o) {
        // TODO Auto-generated method stub
        return UNITPRICE * o.getQuantity();
    }

    @Override
    _____ calcTotDeliv(Order o) {
        _____
        _____
        _____
        _____
    }

    @Override
    public double calcTotAmount(Order o) {
        // TODO Auto-generated method stub
        return calcTotPrice(o)+calcTotDeliv( o);
    }

}
```

3. Complete the missing parts of the class **Tester** as follows:

3.1 Create an instance (called **o**) of **Order** with name "**Melania Trudeau**", quantity: 100 and distance: 15. (2)

3.2 Create an instance (called **oc1**) of the class **OrderCalculatorImp**. (2)

3.3 Call the appropriate method of **oc1** with **o** to display the total amount. (2)

```
package order;

public class Tester {

    public static void main(String[] args) {
        _____
        _____
        System.out.println("Total amount: "+_____);
    }

}
```